```
// Headless verification of Demo 31's collapse math (Theorem 8)
const Zv=150;
let h=100;
const Zh=()=>Zv+h;
const addressOf=(P)=>[ P[1]*(Zv-Zh())/(P[2]-Zh()), P[0]*(Zh()-Zv)/(P[2]-Zv) ];
const rayDir=(a)=>[a[1],-a[0],h];
const cr=(A,B,C,Dd)=>((C-A)*(Dd-B))/((C-B)*(Dd-A));
function rayDistance(a,b){
  const p=[0,a[0]-b[0],0], u=rayDir(a), v=rayDir(b);
  const c=[u[1]*v[2]-u[2]*v[1],u[2]*v[0]-u[0]*v[2],u[0]*v[1]-u[1]*v[0]];
  const n=Math.hypot(...c);
  return n<1e-12?0:Math.abs(p[0]*c[0]+p[1]*c[1]+p[2]*c[2])/n;}
function chainCoeffs(E1,E2){
  const ZH=Zh();
  const Dx=E2[0]-E1[0],Dy=E2[1]-E1[1],Dz=E2[2]-E1[2];
  const p1=-Dx*Zv,p0=-E1[0]*Zv,q1=Dz,q0=E1[2]-Zv;
  const r1=-Dy*ZH,r0=-E1[1]*ZH,s1=Dz,s0=E1[2]-ZH;
  return {a:r1*q0-r0*q1,b:r0*p1-r1*p0,c:s1*q0-s0*q1,d:s0*p1-s1*p0};}
const toScreen=(P)=>[-P[0]*Zv/(P[2]-Zv), -P[1]*Zh()/(P[2]-Zh())];

const E1=[-70,-20,310], E2=[80,30,420];
const COPL=[[-60,-28,310],[10,48,340],[70,114,370],[-20,30,400]];
const PERT=[40,-35,25,-45];
const pts=(cop)=>cop?COPL:COPL.map((P,i)=>[P[0],P[1]+PERT[i],P[2]]);

function metrics(hh,cop){
  h=hh;
  const ads=pts(cop).map(addressOf);
  const lam=cr(ads[0][0],ads[1][0],ads[2][0],ads[3][0]);
  const mu =cr(ads[0][1],ads[1][1],ads[2][1],ads[3][1]);
  let gap=1e18;
  for(let i=0;i<4;i++)for(let j=i+1;j<4;j++)
    gap=Math.min(gap,rayDistance(ads[i],ads[j]));
  const S=[],N=40;
  for(let i=0;i<=N;i++){const u=i/N;
    S.push(toScreen([E1[0]+u*(E2[0]-E1[0]),E1[1]+u*(E2[1]-E1[1]),E1[2]+u*(E2[2]-E
  const A=S[0],B=S[N],L=Math.hypot(B[0]-A[0],B[1]-A[1]);
  let bow=0;
  for(const p of S)
    bow=Math.max(bow,Math.abs((B[0]-A[0])*(A[1]-p[1])-(A[0]-p[0])*(B[1]-A[1]))/L)
  return {lam,mu,fuse:Math.abs(lam-mu),gap,bow};
}

// 1) c = Dz*h identically across separations
```

```
const Dz=E2[2]-E1[2]; let w=0;
for(const hh of [0.5,1,2,5,10,25,50,100]){
  h=hh; const M=chainCoeffs(E1,E2);
  w=Math.max(w,Math.abs(M.c-Dz*hh));
}
console.log("c = Dz*h exact, worst dev        :", w.toExponential(2));

// 2) coplanar fusion: |lam-mu| -> 0 linearly (log-log slope ~ 1)
const f1=metrics(1,true).fuse, f05=metrics(0.5,true).fuse;
console.log("coplanar |lam-mu| at h=100,1,0.5 :",
  metrics(100,true).fuse.toFixed(4), f1.toExponential(2), f05.toExponential(2));
console.log("log-log slope (expect ~1)        :",
  (Math.log(f1/f05)/Math.log(2)).toFixed(3));

// 3) mu is EXACTLY h-independent (t_i = h*X_i/(Z_i-Zv): scale-invariant cr)
const mus=[100,10,1,0.5].map(hh=>metrics(hh,true).mu);
console.log("mu across h (constant)           :", mus.map(m=>m.toFixed(9)).join("
console.log("coplanar limit: lam(0.5) vs mu   :",
  metrics(0.5,true).lam.toFixed(6), mus[0].toFixed(6));

// 4) generic points: |lam-mu| plateaus at a nonzero limit
console.log("generic |lam-mu| at h=100,1,0.5  :",
  metrics(100,false).fuse.toFixed(4),
  metrics(1,false).fuse.toFixed(4), metrics(0.5,false).fuse.toFixed(4));

// 5) concurrence re-forms: min pairwise ray gap -> 0; rays approach O
console.log("min ray gap at h=100,10,1,0.5    :",
  [100,10,1,0.5].map(hh=>metrics(hh,true).gap.toFixed(3)).join(" "));
h=0.5;
let wo=0;
for(const a of pts(true).map(addressOf)){
  const s=a[0],t=a[1];      // dist of ray to O=(0,0,Zv)
  wo=Math.max(wo, Math.abs(s)*Math.hypot(h,t)/Math.hypot(t,s,h));
}
console.log("max dist ray->O at h=0.5         :", wo.toFixed(3));

// 6) the bow dies with h
console.log("bow at h=100,10,1,0.5            :",
  [100,10,1,0.5].map(hh=>metrics(hh,true).bow.toFixed(3)).join(" "));
```